D2 — Attack Surface and Working Harness

1. Attack surface analysis (Track B)

Entry points in libheif that parse untrusted input. The harness calls `heif_context_read_from_memory` and then decodes the image.

#	Entry point	Input it parses	Guard or condition before it is reachable	Harnessed?
1	ISOBMFF box parser	Box headers and sizes	A valid top-level `ftyp` box and nested box lengths	Yes (`file_fuzzer`)
2	Item/property parser	Item references, properties, extents	A valid `meta` box with consistent item tables	Yes (`api_fuzzer`)
3	Image handle decode	Pixel payload and metadata	A supported codec and a decodable item	Yes (`file_fuzzer`)
4	Color conversion	Color profiles and matrices	A decoded image with a color profile	Yes (`color_conversion_fuzzer`)
5	Tile grid decode	Tile offsets and sizes	A grid item with valid tile extents	Yes (`tile_fuzzer`)
6	Sequence/track decode	Track boxes and samples	A sequence track with valid sample tables	Yes (`sequence_fuzzer`)
7	Box-level parser	Individual boxes	Direct parse of one box; no container needed	Yes (`box_fuzzer`)
8	Encoder path	Encoding parameters	An encoder API call, not the decode path	Yes (`encoder_fuzzer`)

All decode entry points are reachable from the harnessed memory-decode API. The encoder path is built and run for coverage, but it is outside the D1 decode

claim.

2. Programs and test cases

Program	Purpose	Build status
file_fuzzer	Decode a HEIF/HEIC/AVIF file	Builds
api_fuzzer	Exercise extended API surfaces	Builds
box_fuzzer	Parse individual boxes	Builds
color_conversion_fuzzer	Color conversion paths	Builds
tile_fuzzer	Tile grid decoding	Builds
sequence_fuzzer	Image sequences	Builds
encoder_fuzzer	Encoding paths	Builds

No fuzzer failed to build. All use the upstream targets unchanged. The seed corpus is the upstream `fuzzing/data/corpus` directory.

3. Repository

https://github.com/lordzadd/libheif-fuzz-d2

4. Exact commands to run the harness

```
git clone https://github.com/lordzadd/libheif-fuzz-d2.git
cd libheif-fuzz-d2
python3 harness/run.py
```

The harness prints a JSON summary with `total_crashes` and writes `results/fuzz_results.csv`.

AI Use Appendix

I used the opencode AI coding assistant to draft the harness, the build script, and this document. I reviewed and edited the output, then tested a clean clone on macOS and on Ubuntu 24.04 with clang, CMake, and make.